



Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

Design and Implementation of an IoT-Enabled Rover for Person Following

By

Hamza Bodair & Mira Hindawi

Supervised by

Prof. Esam Qaralleh

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE

in

Computer Engineering

at

PRINCESS SUMAYA UNIVERSITY FOR TECHNOLOGY

Amman, Jordan

Spring 2025/2026

This is to certify that I have examined
this copy of an engineering documentation by

Hamza Bodair & Mira Hindawi

And have found that it is complete and satisfactory in all respects,
And that any and all revisions required by the final Examining Committee have been
made

Prof. Esam Qaralleh

Dr. Awos Kanan

Acknowledgments

We would like to thank our supervisor, Prof. Esam Qaralleh, for his unconditional support and guidance. Providing us with continuous feedback when needed and mentoring us throughout the project.

We are grateful to our families for their endless support and encouragement throughout our academic journey.

Special thanks to the previous colleagues, Ahmad Mwalla, Rama Abdelhadi, and Mohammad Fayoumi, who developed the rover structure as part of their Senior Design Project.

Finally, we would like to express our appreciation to Princess Sumaya University for Technology and the King Abdullah II School of Engineering for providing a suitable academic environment and laboratories with the resources required to make this project possible.

Generative AI Usage Declaration

We, Hamza Bodair & Mira Hindawi, hereby declare that:

- I. We are solely responsible for the content provided in this work, including any AI-generated content.
- II. We are solely responsible for any copyright issues that could result from using generative AI tools or otherwise.
- III. All the Generative AI tools we have used during the preparation of this work are listed in the table below, and failure to disclose any usage is considered academic misconduct or plagiarism that could result in degree revocation.

Tool	Extent of use	Purpose of Use
ChatGPT-5.5	Section 1.6	Creating Figure 1-1 with the prompt provided by team members
Claude Sonnet 4.6	Chapter 2	Proofreading and citations
Gemini 3	Section 3.4	Helped with the research of different components
ChatGPT-5.5	Section 3.5	Creating Figure 3-1 with the prompt provided by team members
Claude Sonnet 4.6	Dashboard	Rover's dashboard was implemented with the help of Claude

Signature:

Hamza Bodair

Mira Hindawi

Date

12/5/2026

Abstract

Autonomous mobile robots are a significant capability in services, such as warehouse logistics and assistive care. Constructing such systems should not only perceive and interpret a dynamic environment in real time, but also act intelligently without compromising safety, connectivity, or remotely observable states. This project helps overcome that challenge by designing and implementing an IoT-enabled mecanum-wheel rover that autonomously follows a designated person.

The system consists of two computation layers. A Raspberry Pi, acting as the brain, runs a YOLOv8n-based object detection pipeline with MediaPipe hand-gesture recognition to identify, track, and continuously follow a target person. The rover implements a contact-free gesture-based locking protocol, in which the user raises a peace sign to start tracking and an open palm to release it. The low-level motor control, obstacle detection using three ultrasonic sensors, NeoPixel LED status indicator strip, and battery telemetry are handled by an Arduino Mega. The two boards communicate via a serial link, with the Raspberry Pi converting high-level tracking decisions into directional commands which the Arduino executes in real time.

On the connectivity side, the rover is integrated with a cloud-hosted IoT dashboard, which provides live video streaming and manual controls. The mecanum-wheel drive platform enables the rover to move in any direction; rotate and strafe, providing it with excellent maneuverability in tight spaces. Obstacle avoidance is autonomously controlled, stopping the rover when an obstacle is detected within 38 cm. The project demonstrates the possibility of using embedded hardware to implement a real-time person-following rover.

Table of Contents

List of Tables	v
List of Figures	vii
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	2
1.3 List of Design Requirements	3
1.4 List of Design Constraints	4
1.5 List of Engineering Standards	4
1.6 Design Overview	4
1.7 Load Distribution	6
2 Background and Literature Review	7
2.1 Background	7
2.2 Literature Review	8
2.3 Summary	10
3 Design	11
3.1 Analysis of Design Requirements	11
3.2 Analysis of Design Constraints	13
3.3 Discussion of Engineering Standards	14
3.4 Different Design Approaches	14
3.5 Design Development	25
4 Results	36
4.1 Prototype Setup	36
4.2 Testing	42
4.3 Prototype Testing Results Discussion	46
4.4 Validation of Design Requirement within the Realistic Constraints	52
5 Conclusion and Future Work	53
5.1 Conclusion	53
5.2 Future Work	54
References	55
Appendices	56
Appendix A: Offering Template	56
Appendix B: Code	58

List of Tables

Table 1-1: Load Distribution	6
Table 3-1: Advantages and Disadvantages of Lightweight Deep-Learning Detectors	15
Table 3-2: Advantages and Disadvantages of YOLO-Family Single-Stage Detectors	15
Table 3-3: Advantages and Disadvantages of Depth-Based Detection	15
Table 3-4: Advantages and Disadvantages of Detection-Only Tracking	16
Table 3-5: Advantages and Disadvantages of Tracking-by-Detection	16
Table 3-6: Advantages and Disadvantages of Model-Based Predictive Tracking	17
Table 3-7: Advantages and Disadvantages of Automatic Target Selection	17
Table 3-8: Advantages and Disadvantages of Dashboard-Based Target Selection	18
Table 3-9: Advantages and Disadvantages of Gesture-Based Target Locking	18
Table 3-10: Advantages and Disadvantages of Ultrasonic Sensors	19
Table 3-11: Advantages and Disadvantages of LiDAR-Based Obstacle Detection	19
Table 3-12: Advantages and Disadvantages of Vision-Based Obstacle Detection	19
Table 3-13: Advantages and Disadvantages of MQTT Communication	20
Table 3-14: Advantages and Disadvantages of HTTP REST Communication	20
Table 3-15: Advantages and Disadvantages of a Single Battery Pack Power Supply	21
Table 3-16: Advantages and Disadvantages of a Split Power Supply	21
Table 3-17: Advantages and Disadvantages of Dedicated Motor Controllers	22
Table 3-18: Advantages and Disadvantages of L298N Dual H-Bridge Motor Drivers	22
Table 3-19: Advantages and Disadvantages of Differential Drive	23
Table 3-20: Advantages and Disadvantages of Mecanum Wheel Drive	23
Table 3-21: Advantages and Disadvantages of Arduino Uno/Nano	24
Table 3-22: Advantages and Disadvantages of Arduino Mega 2560	24
Table 3-23: Rover State Indicator Behavior	26
Table 3-24: Battery Monitoring Circuit Parameters	27
Table 3-25: Serial Communication Protocol Between Raspberry Pi and Arduino	30

Table 4-1: Main 4S2P Battery Pack Capacity, Energy, and Estimated Runtime Calculation	36
Table 4-2: Raspberry Pi Power Bank Capacity, Energy, and Estimated Runtime Calculation	37
Table 4-3: Person Detection and Lock Acquisition Test	42
Table 4-4: Person Following at Various Speeds Test	43
Table 4-5: Rotation and Lateral Tracking Test	43
Table 4-6: Obstacle Avoidance Test	44
Table 4-7: Target Loss and Recovery Test	44
Table 4-8: Manual Override Through Dashboard Test	45
Table 4-9: Battery Life Test	45
Table 4-10: IoT Telemetry and Dashboard Communication Test	46
Table 4-11: Validation of Requirements and Design Constraints	52

List of Figures

Figure 1-1: Overall System Design of the IoT-Enabled Person-Following Rover	5
Figure 3-1: High-Level System Architecture of the Rover System	25
Figure 3-2: Battery voltage divider circuit connected to Arduino Mega analog input A0	27
Figure 3-3: Person Following Rover (Hardware Schematic)	28
Figure 3-4: Main Software Control Flow of the Person-Following Rover	33
Figure 3-5: Horizontal Deadzone Used for Target Centering	35
Figure 4-1: Final Assembled Prototype of the Person-Following Rover	39
Figure 4-2: IoT Dashboard Interface containing Telemetry	41
Figure 4-3: Camera Detection and Tracking Output	47
Figure 4-4: MediaPipe Hand Landmark Detection	47
Figure 4-5: Target Visibility and Frame-Margin Conditions with Corresponding Rover Commands	48
Figure 4-6: IoT Dashboard Interface During Rover Operation	51

1 Introduction

Rapid development of embedded computing and computer vision has made autonomous mobile robots practical in human environments without constant manual control. Such environments include service areas, campuses, warehouses, and hospitals [1][2]. What is of particular interest, as an engineering problem, is that it requires combining different disciplines into a single system that must work reliably within the physical and computational constraints of a mobile platform. The main goal of this project is to design and implementation of a system that includes an IoT-enabled mecanum-wheel rover that autonomously identifies and follows a specific person using onboard camera-based perception, locks onto its target with a specific hand gesture, avoids obstacles without external intervention, and is remotely observable and controllable via a cloud-hosted dashboard.

1.1 Motivation

The capability of a robot to autonomously track a person without manual control is a feature with wide practical applications. It can be found in luggage carriers, medical equipment trolleys that follow nurses through hospital corridors, and surveillance platforms. In real-world scenarios, the robot can lose the target temporarily due to a sudden change of direction or confusion with other people in the frame. Combining deep-learning detection and tracking enhances continuity and minimizes unstable motion [3].

Camera-based perception offers a cost-efficient and flexible alternative to specialized sensors, but it still requires robust detection and tracking to operate reliably. However, real-time vision algorithms have a high computational cost, and embedded systems must balance accuracy, latency, and power consumption [4]. Another motivation is to integrate autonomous following with IoT capabilities. Complete autonomy is useful, but remote control and manual override are more practical and safer. This is why the project is not limited to person detection; it also includes communication with a dashboard, telemetry support, and remote control as needed.

1.2 Objectives

The primary objective of this project is to design and develop a camera-based person detection and tracking system for an autonomous rover that follows without colliding with any obstacles, connected via cloud through a remote IoT dashboard.

To obtain this objective, the project will be designed based on the following sub-objectives:

- Implement a real-time vision pipeline that can identify and track a specified individual using YOLOv8n and the ByteTrack multi-object tracker.
- Create a gesture-based locking system based on MediaPipe hand landmark detection, to enable the user to lock and unlock the system.
- Combine the camera module and the rover motion control system so that the rover can follow autonomously.
- Implement obstacle detection and avoidance to ensure safe operation during movement.
- Build an IoT communication layer for remote monitoring, telemetry, and manual override through a web dashboard.
- Enhance system performance to meet the needs of real-time operation and energy efficiency.

1.3 List of Design Requirements

1.3.1 Real-time Person Tracking:

- The rover shall process video feed from an onboard camera to detect and track a designated person.
- The system shall be able to follow the target at a speed of at least 0.5 m/s with minimal delay.
- The design shall utilize image processing techniques (e.g., OpenCV) to identify and track objects in real time.

1.3.2 IoT Communication:

- Data such as the rover's current location, target position, and sensor readings shall be sent to a cloud server.
- The system shall support remote control and monitoring through a web or mobile interface.
- The IoT platform shall allow users to receive alerts or control the rover manually if needed (e.g., via MQTT or HTTP protocols).

1.3.3 Obstacle Detection and Avoidance:

- The rover shall detect and avoid obstacles using sensors such as ultrasonic or LiDAR.
- The system shall prioritize safety by stopping or navigating around obstacles while tracking the target.

1.3.4 Energy Efficiency and Battery Life:

- The rover shall operate for a minimum of 1 hour on battery power.
- The design shall optimize power consumption by using low-power sensors and efficient processing algorithms.

1.4 List of Design Constraints

- The cost of the design shall be less than 500 JD.
- The choice of batteries (e.g., lithium-ion, nickel metal hydride) should consider environmental impact, including toxicity and recycling options. Rechargeable and long-life batteries reduce waste and the need for frequent replacements.

1.5 List of Engineering Standards

- IEEE 802.11 – Wi-Fi Standards
- IEC 62133 – Safety Requirements for Portable Battery Systems

1.6 Design Overview

The rover system consists of three main subsystems: vision perception, motion control with obstacle detection, and IoT communication.

The Raspberry Pi runs the vision subsystem. It processes the frames captured by the camera, detects individuals, follows the chosen target, and decides whether to move the rover forward, rotate, or stop it. A specific hand gesture is used to lock the detected target, thus ensuring that the rover follows the target rather than randomly selecting any person in the camera frame.

The Arduino Mega is responsible for motion control. The Raspberry Pi transmits high-level commands via serial to the Arduino, which interprets them as motor movements. This simplifies system management, as the Raspberry Pi focuses on vision and decision-making, while the Arduino handles low-level motor control and hardware interaction. The obstacle detection uses ultrasonic sensors to scan the surroundings of the rover. Obstacle detection takes over person-following/manual commands in case a collision threat is detected, since safety is a priority.

IoT communication offers remote monitoring and control features. The user is able to observe the rover on the dashboard, receive status updates, view the camera stream, and alternate the rover's control between autonomous and manual.

Figure 1-1 presents a conceptual overview of the IoT-enabled person-following rover.



Figure 1-1: Overall System Design of the IoT-Enabled Person-Following Rover

1.7 Load Distribution

Table 1-1: Load Distribution

Task	Mira	Hamza	From/To
Introduction	40%	60%	Nov
Background & Literature Review	30%	70%	Nov–Dec
Initial Designs & Sketching	70%	30%	Nov–Dec
Design Approaches & Justifications	50%	50%	Jan
System Architecture & Pseudo-code	40%	60%	Mar
Arduino Code (motor, sensors, LEDs)	30%	70%	Nov–May
Raspberry Pi Code (vision, dashboard)	40%	60%	Nov–May
Hardware Assembly & Schematic	60%	40%	Nov-Apr
Testing & Results	50%	50%	Mar-May
Validation Table & Conclusion	70%	30%	Apr-May
Final Documentation & Proofreading	50%	50%	Apr-May

2 Background and Literature Review

2.1 Background

Autonomous human-following mobile robots have been actively studied over the past twenty years [5], but it has recently become practically feasible on low-cost platforms [6]. This was made possible by the availability of deep learning-based object detectors that can run fast enough on embedded processors, the advancement of multi-object tracking algorithms that can maintain identity across frames, and the increasing availability of inexpensive single-board computers that are powerful enough to run heavy processes in real time.

The platform used in this project is a mecanum-wheel rover. In contrast to the traditional wheels, mecanum wheels enable the rover to move forward, backward, sideways, and rotate by controlling the direction of each wheel independently. For a person-following rover, this motion is helpful since the robot can change its position gradually and smoothly as the target moves around.

The perception part of the system is based on YOLOv8n. YOLO-family detectors directly predict bounding boxes and class probabilities using pixel values in a single pass, which is much faster than the two-stage detectors like Faster R-CNN [7] but remains competitive in accuracy. The nano variant, YOLOv8n, which is used in this project, is specifically designed to be edge-deployed, trading off some detection accuracy for faster processing on embedded devices such as Raspberry Pi.

Frame-to-frame identification of individuals is preserved using ByteTrack, a multi-object tracking algorithm that associates detected bounding boxes across consecutive frames. The special feature of ByteTrack is that it measures both high-confidence and low-confidence detections during the association step, helping it preserve track continuity through brief occlusions and detection failures that occur when a person turns sideways or partially steps behind an obstacle [8].

MediaPipe Hands is a lightweight hand landmark detection library developed by Google to handle gesture recognition. MediaPipe Hands approximates the 3D locations of 21 keypoints on each hand using a single RGB frame [9]. The landmarks used in this project to categorize two gestures include a peace sign (index and middle fingers extended, ring and pinky folded) for target locking, and an open palm (four or more fingers extended) for target unlocking.

The IoT layer used to connect the rover to a cloud-hosted dashboard via HTTP REST API calls over a Wi-Fi connection. The Raspberry Pi is the IoT client, which transmits heartbeat messages and telemetry data (battery percentage, CPU temperature), pushes live JPEG video frames, and polls for pending manual override commands. Absolute rover location was not implemented because the rover is mainly intended for indoor use, where GPS signals are weak and unreliable.

2.2 Literature Review

Person-following mobile robots have been studied on various perceptions, tracking, control, and embedded hardware combinations. The studies show that the challenging part of person following is not only detection but also system integration, real-time processing, target identity, and hardware limitations, all of which should be considered when designing such systems.

Eirale et al. [2] give a detailed survey of human following and guidance systems in autonomous mobile robots. They address common system elements such as perception, tracking, control, and navigation, and emphasize the challenges posed by occlusion, dynamic human motion, changing environments, and real-time constraints. These challenges are directly applicable to this project as the rover will need to follow a selected person indoors and deal with situations where the target is lost, and multiple people are present.

In [3] Condés et al. introduce a mobile robot system for robust person identification and following using deep learning and optical tracking. Their results demonstrate the advantage of detection and tracking over detection only for stability. This directly aligns with the design decision in this project, which is the combination of YOLOv8n detection with ByteTrack to ensure the detected target is maintained across frames, and the movement is not too unstable.

For an orchard mobile robot, Huang et al. [4] suggest a human-following approach based on YOLO and KCF tracking. While the application is different, the results of the two show the necessity of incorporating a tracker with an object detector in changing environments, as tracking can mitigate the impact of temporary target loss and background complexity, which is why ByteTrack is used in this project.

Kamath and Renuka [10] explore human following on a Raspberry Pi using the MobileNet-SSD, showing the accuracy vs. speed trade-offs on embedded hardware. The study is directly applicable, since the rover used in this project is also based on a Raspberry Pi, and the frame resolution, model selection, and detection frequency have to be optimized to ensure real-time response within the hardware's capabilities.

Rakprayoon et al. [5] refer to a cognitive robotic system for human following, where the perception of the target is closely related to the movement behavior. This is important for this project because not only does the Raspberry Pi detect the target, but it also translates the target's location in the frame into commands for the Arduino to control the robot.

Boschi et al. [6] introduce a cost-effective person-following system for assistive unmanned vehicles based on deep learning inference at the edge on a single-board computer. Their work is a key foundation for this project's aim of developing reliable person-following systems on low-cost embedded hardware, and their optimization results for the model deployment on the edge directly influenced the pipeline design used here.

Redmon et al. [7] proposed the YOLO detection method, which is a one-pass detection method, and is very suitable for real-time applications. This project is based on that concept, using YOLOv8n, which offers a good trade-off between speed and accuracy.

2.3 Summary

An analysis of the literature indicates that person-following robots are feasible, but difficult to implement, especially on low-cost embedded platforms. Real-time processing, stable target tracking, safe navigation, and efficient use of computational resources are all commonly discussed features. The major drawbacks mentioned are limited frame rates on low-end devices, loss of targets when they are occluded, problems with target identity in cluttered scenes, and unreliable performance in varying environments.

The studies reviewed demonstrate that detection and tracking are more reliable than detection only, which is why YOLOv8n is implemented with ByteTrack in this project. Gesture-based locking using MediaPipe also solves the problem of target selection by allowing the intended persons to lock themselves directly to the rover, lowering the risk of following the wrong person in multi-person scenes.

The majority of related systems focus on either the accuracy of perception or navigation behavior. There are fewer systems that aim to create a complete low-cost embedded rover that integrates person detection, target locking, motion control, obstacle detection, IoT monitoring, and manual override on one platform. This project will fill that gap by introducing a practical person-following rover consisting of a Raspberry Pi 5, an Arduino Mega, camera-based tracking, ultrasonic safety sensing, and dashboard-based IoT control.

3 Design

3.1 Analysis of Design Requirements

3.1.1 Person Tracking in Real-Time

The rover should be able to detect and track a specific individual in real time based on a live camera feed. The Raspberry Pi 5 is selected for its ability to run the following on a single embedded platform: camera capture, person detection and tracking, gesture recognition, and serial command output. The camera runs in a separate thread and pulls from the capture buffer to ensure the main loop uses the newest available frame, maintaining low latency. YOLOv8n is used at a lower input resolution for person detection, along with ByteTrack, which maintains a constant target identity across frames to prevent unstable behavior. A hand gesture locks the rover onto a desired individual, minimizing the chance of following the wrong person when more than one person is in sight.

3.1.2 IoT Communication

To support remote monitoring and control, the rover must provide IoT communication, which requires the design of a dashboard communication layer for the Raspberry Pi software. The layer is implemented on a background thread, ensuring that network delays do not block the camera loop or delay motor commands. It transmits telemetry and live video and includes a manual override feature that lets you switch from autonomous following to direct control via the dashboard when necessary.

Safety-critical behavior is entirely local. Obstacle detection is handled by the Arduino, independent of the dashboard's state, and the Raspberry Pi's state machine handles stopping when the target is lost. This means the rover will not be exposed to any danger even if the Wi-Fi connection is lost, which is necessary for a mobile system operating in an environment with unreliable network conditions.

3.1.3 Obstacle Detection and Avoidance

The rover should be able to identify and avoid any obstacles on its way to its target. This added a safety layer to the design, implemented on the Arduino, so that the system could operate without the vision pipeline. To achieve this, ultrasonic sensors are used. If the sensor detects an object, the Arduino overrides the command being executed regardless of what the Raspberry Pi has requested. This ensures that the obstacle avoidance always takes precedence over the following behavior. Separating obstacle detection processing from the vision pipeline reduces processing load on the Raspberry Pi and ensures the safety check processing continues even when the vision loop is temporarily occupied.

3.1.4 Energy Efficiency and Battery Life

The rover should be able to work for at least one hour on a single charge. This influenced both the hardware power architecture and the software performance settings.

On the hardware side, the power system is split into two parts: a 4S2P (4 cells in series, 2 parallel packs) Li-ion pack supplying the Arduino, motors, and system sensors, and a dedicated power bank powering the Raspberry Pi. This separation ensures that any voltage drop during motor operation does not affect the Raspberry Pi's power supply rail.

On the software side, the vision pipeline was intentionally not allowed to run at full settings. To lower CPU load and heat, which directly affect battery draw, the camera resolution is reduced, the smallest possible YOLO model variant is used, and skipping frames between detections is enabled.

3.2 Analysis of Design Constraints

3.2.1 Economic constraint

The project's cost limit of 500 JD influenced component selection and the design approach. Rather than a Jetson Nano (which would have offered GPU-accelerated inference), a Raspberry Pi 5 was chosen as the compute platform. The Pi 5's better CPU than its predecessors allows it to run YOLOv8n at reasonable speeds without a GPU. The choice of L298N dual H-bridge motor drivers over more advanced motor controllers was based on their low cost, wide availability, and ability to drive PWM-controlled DC motors at the currents the rover will need. The HC-SR04 ultrasonic sensors were chosen over LiDAR for the same reason.

3.2.2 Environmental constraint

The lithium-ion 18650 cells used in the main battery pack pose thermal and chemical risks in the event of overcharging, over-discharging, or short-circuiting. The design shows this by measuring the pack voltage in firmware and displaying it via the dashboard. Physically, the cells are contained in a special container that keeps them separated, and the pack is never charged while in the rover. The decision to use 18650 cells rather than lithium-polymer pouches also minimizes the risk of puncture.

3.2.3 Manufacturability and sustainability constraint

All components in this design are standard, off-the-shelf parts available from electronics distributors with no lead time or minimum order requirements. Some of the most well-documented and supported components in the academic robotics community are the Arduino Mega, Raspberry Pi 5, L298N drivers, and HC-SR04 sensors. This implies that any failed part can be replaced without the need to fabricate it or any special tooling.

3.3 Discussion of Engineering Standards

Engineering standards support safety, reliability, and compatibility. Two standards are considered important for this project:

3.3.1 IEEE 802.11 (Wi-Fi Standards)

Wireless communication between the Raspberry Pi 5 and the cloud-hosted dashboard is governed by IEEE 802.11. The reliability of the standard supports the HTTP-based command polling, telemetry transmission, and live video streaming. In a low-cost, widely applicable project, it is important to use a standardized wireless protocol.

3.3.2 IEC 62133 (Safety Requirements for Portable Battery Systems)

IEC 62133 addresses safety considerations for lithium-ion cells used in the rover's main battery pack. Pack voltage is measured by the Arduino firmware through a resistive divider circuit, averaging the readings over six ADC samples, and mapping the result to a state-of-charge percentage sent to the dashboard every one minute.

3.4 Different Design Approaches

Several design alternatives were considered before settling on the final system architecture. This section presents the main decision points, the options evaluated for each, and the reasoning behind each choice.

3.4.1 Person Detection Approach

Three approaches were considered for detecting people in the camera feed.

Alternative A: Lightweight Deep-Learning Detectors (example, MobileNet-SSD)

Small neural networks that can be used to detect objects with minimal computational resources. They are commonly used on embedded devices where processing power, memory, and energy consumption are limited.

Table 3-1: Advantages and Disadvantages of Lightweight Deep-Learning Detectors

Advantages	Disadvantages
• Low computational load • Reduced power usage • Faster inference on CPU-based systems	• Lower detection accuracy in complex scenes • Dependent on indoor lighting variations

Alternative B: YOLO-Family Single-Stage Detectors (Compact Variants)

YOLO-family detectors are real-time object detection models, where objects are detected and their bounding boxes are predicted in a single pass through the network.

Table 3-2: Advantages and Disadvantages of YOLO-Family Single-Stage Detectors

Advantages	Disadvantages
• Improved accuracy, speed trade-off for real-time identification • More robust in indoor environments • Native integration of ByteTrack	• Higher computational requirements • Needs to optimize according to real-time and energy requirements

Alternative C: Depth-Based Detection (RGB-D Camera)

Helps separate the person from the background and estimate distance, allowing detection of targets based on their distance and shape.

Table 3-3: Advantages and Disadvantages of Depth-Based Detection

Advantages	Disadvantages
• Less sensitive to the changes in light • Direct distance estimation to the target	• Greater cost of hardware than of monocular cameras • Increased power consumption

Design Justification: Compact YOLO-based detectors were selected for its accuracy and speed and because they provide more reliable person detection while remaining suitable for embedded use.

3.4.2 Tracking Strategy

Three methods were considered to keep the target identity consistent across all frames.

Alternative A: Detection-Only Tracking Motion

Per-frame detection is used to generate the commands without any tracker.

Table 3-4: Advantages and Disadvantages of Detection-Only Tracking

Advantages	Disadvantages
• Implementing is simple • Very low processing overhead	• Unsmooth motion caused by detection noise • Target loss during occlusions or missed detections

Alternative B: Tracking-by-Detection (example, ByteTrack)

Combination of an object detector and a tracking algorithm. The detector detects objects in each frame, and the tracker links detections within a sequence of frames to maintain the identity of the same target in consecutive frames.

Table 3-5: Advantages and Disadvantages of Tracking-by-Detection

Advantages	Disadvantages
• Tracking is smoother • Better continuity and less flicker	• Increased system complexity • Parameters need careful tuning to prevent the tracker from drifting

Alternative C: Model-Based Predictive Tracking (example, Kalman Filter only)

Predicts the direction in which the target is going to move from previous movements. It can be useful for short time gaps in detection, but human movement is unpredictable, affecting accuracy.

Table 3-6: Advantages and Disadvantages of Model-Based Predictive Tracking

Advantages	Disadvantages
• Predictive capability during short detection gaps • Low computational overhead	• Performance declines due to sudden human movement • Requires accurate assumptions of the motion model

Design Justification: To enhance the stability and tracking continuity, without the computational overhead, ByteTrack was chosen as the most suitable for the project's system design and embedded hardware.

3.4.3 Target Selection Method

Three approaches were considered to decide and identify which person to follow.

Alternative A: Automatic Nearest or Largest Person Selection

Automatically selects a target according to simple visual criteria such as the nearest person to the camera, the largest person in the frame, or the nearest person to the center of the frame.

Table 3-7: Advantages and Disadvantages of Automatic Target Selection

Advantages	Disadvantages
• No gesture from the user is required • Implementation is easy, simple, and fast	• In crowded scenes, the wrong person can be selected • The targets can change in an unpredictable way

Alternative B: Dashboard-Based Target Selection

The user selects the target through the system's remote dashboard.

Table 3-8: Advantages and Disadvantages of Dashboard-Based Target Selection

Advantages	Disadvantages
• Direct user control • Avoids unintentional target selection	• Networks availability • Slower than other techniques • Dashboard availability

Alternative C: Gesture-Based Target Locking

The rover locks onto the person who makes a certain hand gesture in front of the camera.

Table 3-9: Advantages and Disadvantages of Gesture-Based Target Locking

Advantages	Disadvantages
• Target direct control • Doesn't need a network connection • User-friendly	• Reliable hand visibility is required • Additional load on the CPU • Gestures may not be detected

Design Justification: The gesture-based locking has been selected, as it offers the user a direct and simple method of locking themselves, without having to use a remote interface or a network connection. Manual override can always be done using the dashboard, but the initial target selection is always done locally via gesture, keeping the locking behavior independent of network availability.

3.4.4 Obstacle Detection Method

Three sensing approaches were considered for detecting obstacles during the following.

Alternative A: Ultrasonic Sensors (HC-SR04)

Determine the distance by sending out sound waves and measuring the time it takes for the echo to return.

Table 3-10: Advantages and Disadvantages of Ultrasonic Sensors

Advantages	Disadvantages
• Easy integration and low cost • Low power consumption	• Limited accuracy • Affected by surfaces/noise

Alternative B: LiDAR

Laser light is used to measure distance, offering extensive distance measurement.

Table 3-11: Advantages and Disadvantages of LiDAR-Based Obstacle Detection

Advantages	Disadvantages
• High-precision obstacle detection • Supports advanced avoidance strategies	• Expensive • Increased power and processing needs

Alternative C: Vision-Based Obstacle Detection

Involves using camera images and image-processing or machine-learning methods to detect obstacles.

Table 3-12: Advantages and Disadvantages of Vision-Based Obstacle Detection

Advantages	Disadvantages
• No extra sensors will be needed • The system detects various types of obstacles.	• Computationally expensive • Sensitive to light status • Less reliable for precise distance estimation

Design Justification: Ultrasonic sensors were used because they meet safety requirements while not exceeding the budget or power limitations.

3.4.5 Communication Protocol

Two protocols were considered for dashboard communication.

Alternative A: Message Queuing Telemetry Transport (MQTT)

A lightweight messaging protocol that is widely used in IoT systems. Using a publish-subscribe architecture, devices publish and subscribe to messages via a broker.

Table 3-13: Advantages and Disadvantages of MQTT Communication

Advantages	Disadvantages
• Low message overhead • Suitable for high-frequency telemetry	• Requires a special broker • Adds infrastructure complexity

Alternative B: HyperText Transfer Protocol-Representational State Transfer (HTTP REST)

A very popular communication technique that relies on request/response messages and is widely used in web applications and cloud-based services.

Table 3-14: Advantages and Disadvantages of HTTP REST Communication

Advantages	Disadvantages
• No broker required • Compatible with standard web infrastructure • Simpler to implement and secure with token authentication	• Slightly higher message overhead • Command latency

Design Justification: HTTP REST was chosen because it is easy to integrate with the dashboard and does not require a separate message broker. It can support command polling, heartbeat messages, telemetry upload, and video frame transfer on the same web-based infrastructure.

3.4.6 Power Architecture

Two power supply configurations were considered.

Alternative A: Single Battery Pack

Provides power to all parts of the system, such as motors, sensors, controllers, Arduino, and Raspberry Pi 5.

Table 3-15: Advantages and Disadvantages of a Single Battery Pack Power Supply

Advantages	Disadvantages
• Simpler wiring • Fewer components	• Sudden current intakes cause a voltage drop on the logic rail • Risk of brown-out causing the Raspberry Pi to reset during the motors' operation

Alternative B: Split Supply

Separates the power supply of high-current components from sensitive computing or control electronics. This minimizes the electrical noise and voltage drop in the processor.

Table 3-16: Advantages and Disadvantages of a Split Power Supply

Advantages	Disadvantages
• Isolates the Pi from motor supply noise • Prevents brown-out resets • Each rail is sized according to its actual load	• Requires managing two separate power sources • Slightly more complex wiring

Design Justification: The split power supply was chosen as the electrical motors may cause voltage drops and electrical noise while moving and changing direction. The Raspberry Pi can become unstable, or brown-out resets may occur due to these

disturbances. This design ensures the Raspberry Pi is powered independently of the motor supply, helping it remain stable during rover movements and increasing reliability.

3.4.7 Motor Driver

Two motor driver options were considered for driving the four mecanum wheels.

Alternative A: Dedicated Motor Controllers (example, Cytron MDD10A)

Specialized modules that are used to efficiently control DC motors with current sensing and closed-loop speed control.

Table 3-17: Advantages and Disadvantages of Dedicated Motor Controllers

Advantages	Disadvantages
• Smoother PWM control • Current handling and protection are better	• Significantly more expensive

Alternative B: L298N Dual H-Bridge Modules

Standard H-bridge ICs able to operate two DC motors using PWM direction pins. They are frequently used motor driver modules to control the direction and speed of DC motors.

Table 3-18: Advantages and Disadvantages of L298N Dual H-Bridge Motor Drivers

Advantages	Disadvantages
• Low cost and widely available • Integration is easy and is well-documented	• Voltage drops reduce the motor's effectiveness • No built-in current sensing or closed-loop control

Design Justification: The L298N dual H-bridge modules were chosen, as they are low-cost, widely available, and sufficient for the rover. They are a good solution within the project's budget, as the rover needs only PWM speed control and direction control.

3.4.8 Drive Platform

Two drive configurations were considered for the rover chassis.

Alternative A: Differential Drive

A typical mobile robot design, in which the forward motion is achieved by changing the speed of two primary driving wheels. Easy and reliable, but cannot move sideways without turning first.

Table 3-19: Advantages and Disadvantages of Differential Drive

Advantages	Disadvantages
• Simple mechanical design • Lower cost • Easier to control	• Cannot move laterally without first rotating • Requires wide turns to follow a target if moved sideways • Less maneuverability

Alternative B: Mecanum Wheel Drive

Type of wheels that have angled rollers that enable the robot to move diagonally and sideways without the need to turn.

Table 3-20: Advantages and Disadvantages of Mecanum Wheel Drive

Advantages	Disadvantages
• Full omnidirectional movement • Can follow a target that moves sideways smoothly • Maneuverability in indoor spaces	• More complex • Higher cost • Easily affected by rough surfaces

Design Justification: The mecanum wheels were kept from the previous project, as they offer a smoother maneuverability than a standard differential-drive platform. This is helpful for person-following, as the rover could end up drifting off course if the target moves sideways using traditional wheels.

3.4.9 Microcontroller

Two platforms were considered for the low-level hardware control layer.

Alternative A: Arduino Uno / Nano

Arduino Uno and Nano are small microcontroller boards, widely used for simple embedded control applications.

Table 3-21: Advantages and Disadvantages of Arduino Uno/Nano

Advantages	Disadvantages
• Lower cost • Compact size	• Limited number of digital I/O pins • Limited processing capability

Alternative B: Arduino Mega 2560

A larger microcontroller board with 54 digital I/O pins, 16 analog inputs, and 4 hardware serial ports.

Table 3-22: Advantages and Disadvantages of Arduino Mega 2560

Advantages	Disadvantages
• Sufficient I/O for all peripherals • Well documented and widely supported	• Larger size • Higher cost

Design Justification: The Arduino Mega 2560 was chosen as the rover needs a lot of input and output ports, such as four motor ports, three ultrasonic sensors, a NeoPixel LED strip, a buzzer, battery voltage sensing, and serial communication with the Raspberry Pi. It also enables the rover to process movement commands from the Raspberry Pi faster and locally. So, for this project, the Mega offers a faster, more reliable hardware control solution.

3.5 Design Development

The system is designed with two control layers linked by a serial interface, with a cloud layer on top of both. The Raspberry Pi handles all high-level perception and decision-making, the Arduino handles all hardware interaction, and the dashboard provides remote visibility and control. Figure 3-1 illustrates the connections between these three layers: video flows in through the camera to the Pi; serial commands flow from the Pi to the Arduino; and HTTP traffic flows between the Pi and the dashboard via Wi-Fi.

Figure 3-1 illustrates the high-level system architecture of the rover system.

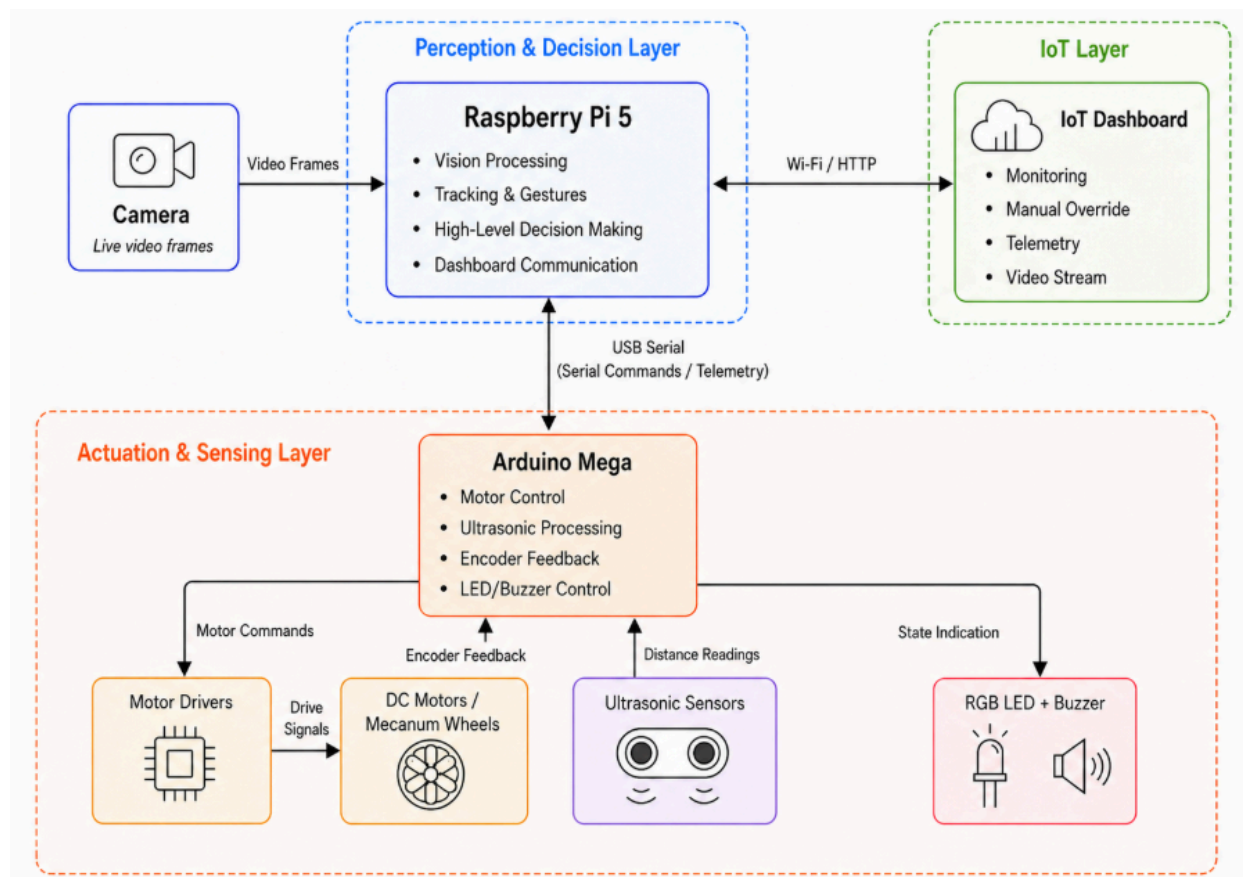


Figure 3-1: High-Level System Architecture of the Rover System

3.5.1 Hardware Architecture

The Arduino Mega 2560 is used to handle all of the system hardware components, which include the following: Four DC motors (front-left, front-right, rear-left, and rear-right) driven by two L298N dual H-bridge modules, with the direction of each motor controlled by a pair of digital output pins and its speed by a PWM enable pin. Three HC-SR04 ultrasonic sensors are connected to dedicated digital I/O pins. The median of three successive ultrasonic pulses is used to reject incorrect readings. A 30-LED NeoPixel strip and a piezo buzzer that provides state feedback are connected to the Arduino, as shown in Table 3-23.

Table 3-23: Rover State Indicator Behavior

Rover State	LED/Buzzer Behavior
FREE	Red moving animation
LOCKED	Solid green LED
LOST	Flashing green LED with repeated double beep
UNLOCKED	Flashing red LED, then returns to FREE

The analog pin reads the battery voltage, which is scaled using a 33 k Ω and 10 k Ω voltage divider to bring it within the Arduino's 5 V ADC range. The system uses a 4S2P 18650 lithium-ion battery pack. The term 4S2P refers to four cells connected in series and two holders connected in parallel. The series connection increases the voltage to approximately 14.4 V nominal and 16.8 V when fully charged, while the parallel connection increases the available capacity and current capability. Because Arduino's analog input can only handle up to 5 V, a voltage divider was used to reduce the rover's battery pack voltage, which can reach approximately 16.8V when fully charged, before it reaches the analog input (A0). The original battery voltage is then calculated in software using the known resistor ratio. Figure 3-2 shows the battery voltage divider circuit connected to the Arduino Mega analog input.

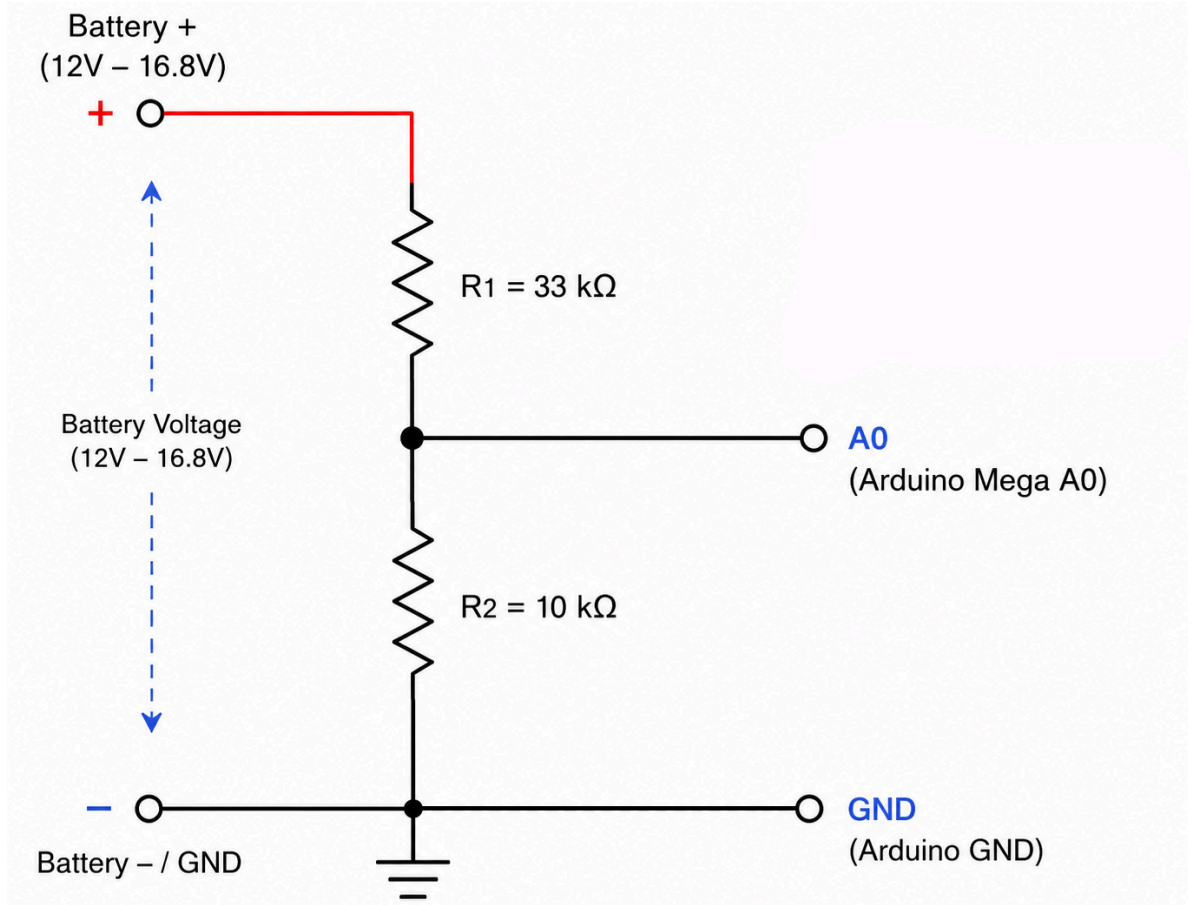


Figure 3-2: Battery voltage divider circuit connected to Arduino Mega analog input A0

The parameters of the battery monitoring circuit are summarised in Table 3-24.

Table 3-24: Battery Monitoring Circuit Parameters

Parameter	Value / Description
Battery pack	4S2P Li-ion batteries: nominal ~14.4 V, fully charged ~16.8 V, empty ~12.0 V
Voltage Divider R1, R2	R1 = 33 kΩ, R2 = 10 kΩ
ADC reference	5 V (Arduino Mega VREF)
Voltage formula [11]	$V_{in} = \frac{ADC}{1023} \times 5 \times \frac{33000+10000}{10000}$

The voltage formula is used to convert the ADC reading from Arduino to the actual battery voltage. The analog reading is between 0 and 1023, where 1023 corresponds to a 5V reference voltage, because the Arduino Mega has a 10-bit ADC. The reduced voltage measured at pin A0 is given by the term $\frac{ADC}{1023} \times 5$. This voltage was reduced by the 33k Ω and 10k Ω , and then multiplied by $\frac{33000+10000}{10000}$ to estimate the voltage of the battery before the voltage divider.

The Raspberry Pi 5 is powered independently by a dedicated power bank and communicates with the Arduino over USB serial at 115,200 baud.

Figure 3-3 shows the hardware schematic of the rover system.

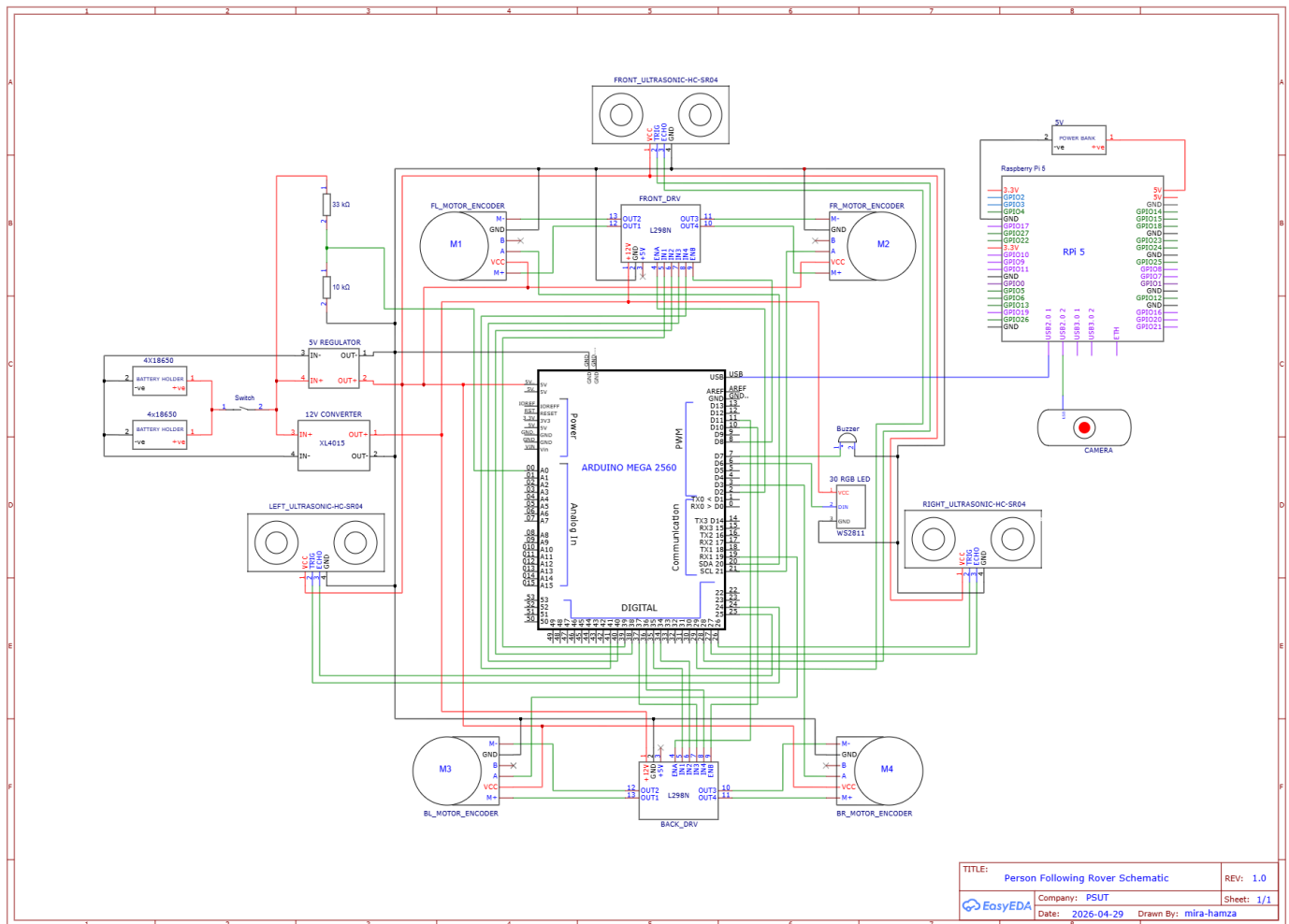


Figure 3-3: Person Following Rover (Hardware Schematic)

3.5.2 Serial Communication Protocol

Each Pi-to-Arduino command is a single ASCII character transmitted at 115,200 baud, with no framing, so the Arduino processes each character as it arrives. This is an asynchronous serial communication protocol in which the Raspberry Pi and Arduino do not have a clock signal of their own; data is sent only when needed.

The messages sent between the Arduino-to-Pi boards are in the fixed format BAT, voltage, and percentage, and are sent at a rate of one message per second. And ultrasonic telemetry in the fixed format SENS, front, left, right, sent every 250 ms of the latest front, left, and right distance readings.

The BAT message relates to the battery monitoring circuit mentioned above, in which the Arduino reads the battery voltage from the voltage divider on analog pin A0. The 'BAT' field identifies the message as battery data, 'voltage' represents the actual voltage value of the system's battery, and 'percentage' represents the calculated percentage value based on the 'voltage' value.

The SENS field identifies the message as ultrasonic sensor data, while front, left, and right represent the measured distances from the three ultrasonic sensors. The entire protocol is defined in Table 3-25.

Table 3-25: Serial Communication Protocol Between Raspberry Pi and Arduino

Command	Char	Direction	Action
Forward	F	Pi → Arduino	Move forward; speed scaled by front ultrasonic distance
Stop	S	Pi → Arduino	Halt all motors
Rotate left/ right (mid)	L/R	Pi → Arduino	Arduino-supported mid-rotation commands; currently, the Pi mainly uses Q/W/E and I/O/P for graduated rotation
Rotate left (low/med/high)	Q / W / E	Pi → Arduino	Graduated counter-clockwise rotation
Rotate right (low/med/high)	I / O / P	Pi → Arduino	Graduated clockwise rotation
Free indicator	A	Pi → Arduino	LED/buzzer → FREE state (red chase animation)
Locked indicator	K	Pi → Arduino	LED/buzzer → LOCKED state (solid green)
Lost indicator	X	Pi → Arduino	LED/buzzer → LOST state (flashing green + double beep)
Unlock flash	U	Pi → Arduino	LED/buzzer → UNLOCK state (red flash, 1.2 s)
Battery telemetry	BAT, V, P	Arduino → Pi	Voltage (2 d.p.) and percentage every 1 second
Ultrasonic telemetry	SENS, F, L, R	Arduino → Pi	Front, left, and right ultrasonic distance readings, sent every 250 ms

3.5.3 Software Architecture

The Raspberry Pi software is a multi-threaded Python program. When the threaded camera mode is activated, a camera reader thread starts, continuously capturing frames and saving only the last available frame. The camera frames are captured at 480×270 resolution, providing sufficient visual information for person detection and gesture recognition while reducing processing load. YOLO detection is then performed at a reduced input size of 288×288 pixels to improve real-time performance on the Raspberry Pi. This minimizes delay, since the main loop will use the latest frame rather than waiting for older buffered frames to be consumed. Person detection, target tracking, gesture recognition, state modes, and movement decision-making are handled in the main loop.

The dashboard functionality runs in separate threads, so network communication does not affect the real-time following loop. These functions are heartbeat transmission, telemetry upload, command polling, and video frame relay. To prevent multiple accesses to the serial port at any given time, a threading lock is employed; to control transitions between autonomous following and manual control, another lock is used.

The autonomous behavior is structured into four primary states: FREE, LOCKED, LOST, and UNLOCKED. In the FREE state, there is no locked target, and the rover remains idle until a valid lock gesture is received. When the system is in a LOCKED state, the rover tracks the chosen target by transmitting movement commands based on the target's relative position in the camera frame. The rover enters the LOST state when the locked target moves out of the frame and the stop command is sent.

The LOST state lasts for 15 seconds. During the first part of this state, the rover stops and waits for the target to reappear. The system returns to the LOCKED state if the target is detected again before the lost timeout runs out. If the target is not detected for 5 seconds, the rover compares the readings from the left and right ultrasonic sensors to determine which wall or obstacle is closer. Then it strafes towards the nearest side closer to the wall, moving out of the path and not blocking the way. If the target is not reacquired within 15 seconds, the system returns to the FREE state.

The system stores an appearance reference of the locked target based on an HSV (Hue, Saturation, Value) color histogram computed from the upper body area, for short-term target recovery. If ByteTrack briefly changes or drops the target ID, the system compares persons with the last known target position, bounding-box height, and HSV histogram similarity before accepting a reacquired target. This helps minimize the risk of switching to another person during short interruptions in tracking or partial occlusions. This method is limited and only used for short-term recovery.

The UNLOCKED state is entered when the unlock gesture is made, and the rover halts and displays the unlock indication before returning to the FREE state. In the case of manual mode being activated, the Raspberry Pi clears the current autonomous tracking state, halts the rover, and temporarily sends manual movement commands, via the dashboard, to the Arduino. Autonomous mode is deactivated while in manual mode until an auto-follow command is sent from the dashboard for safety reasons.

Figure 3-4 illustrates the main software control flow of the person-following rover. The flowchart was created using PlantUML to represent the main software states, decisions, and transitions.

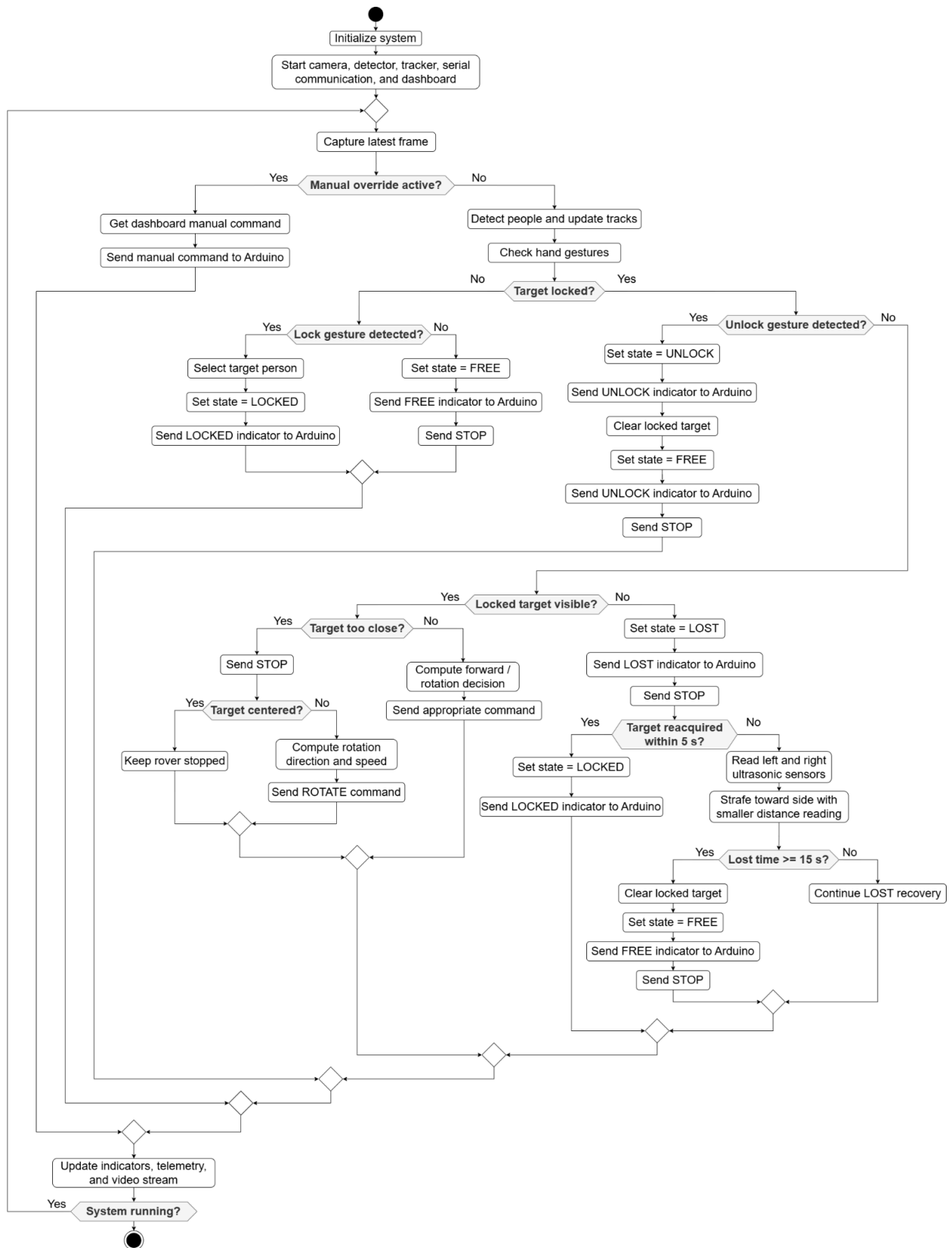


Figure 3-4: Main Software Control Flow of the Person-Following Rover

3.5.4 Control Logic

Detection is done every 1 frame in the LOCKED state and every 3 frames in the FREE state, whereas hand processing is done every 5 frames in the LOCKED state and every 3 frames in the FREE state. These intervals were chosen to reduce CPU load and keep the rover responsive, as running detection on every frame would increase latency and reduce the frame rate available for control decisions. These frame differences indicate that gesture detection is more critical during FREE, when the system is waiting to receive a lock gesture, and that tracking continuity is more important once a target is locked.

After locking, the target bounding box's horizontal center is smoothed with an exponential moving average with a factor of 0.35 to reduce jitter. This value was selected experimentally as a balance between reducing sudden bounding-box noise and keeping the rover responsive to real target movement.

An 18 percent deadzone on both sides of the frame center will prevent the unnecessary micro-rotations when the target is roughly centered. Beyond the deadzone, the horizontal error magnitude is mapped to one of three rotation speeds, LOW, MED, or HIGH, sent to the Arduino as Q/W/E left and I/O/P right. Whenever the target is within the deadzone, forward motion is commanded when the target is centered. When the height of the smoothed bounding box of the target is greater than 190 pixels, the rover is too close, and a stop command is sent. The rotation pulse duration is proportional to the apparent height of the target, between 15 ms with a far target and 250 ms with a close target, and offers proportional correction without an explicit PID implementation. A 180 ms settle period will permit the rover to physically complete the rotation before the next command is issued.

Figure 3-5 shows the horizontal deadzone used for target centering and command selection.

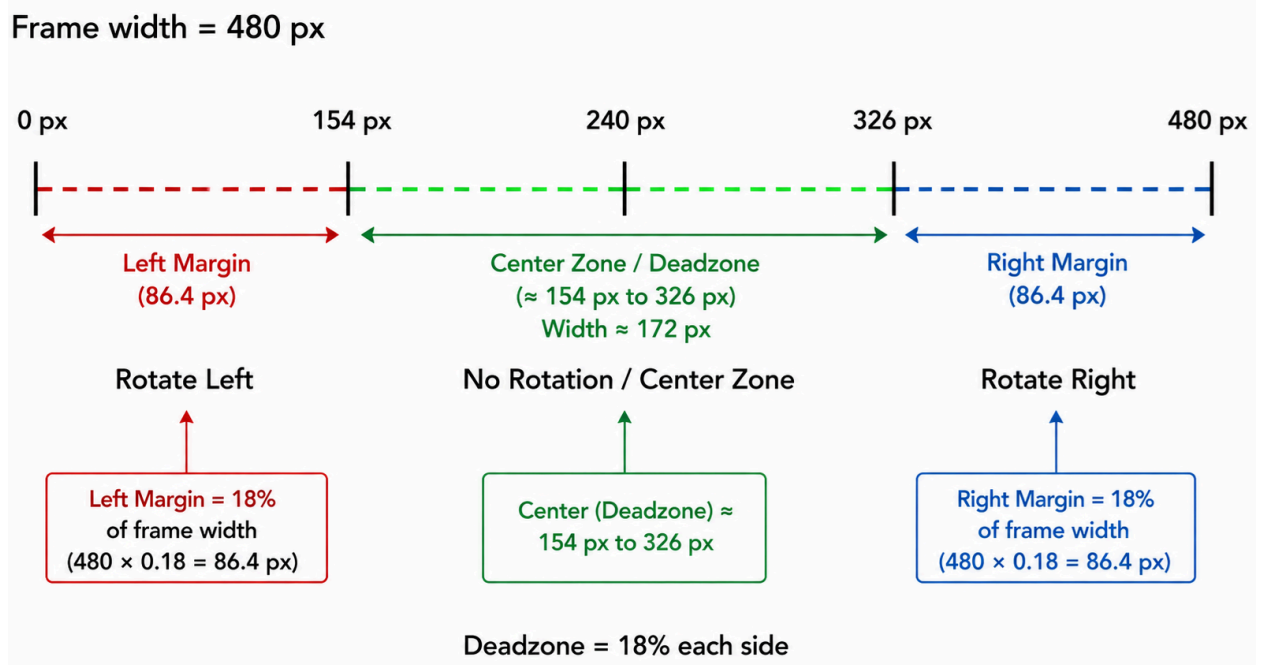


Figure 3-5: Horizontal Deadzone Used for Target Centering

4 Results

4.1 Prototype Setup

4.1.1 Hardware Setup

The prototype is constructed on a mecanum-wheel rover chassis, a continuation of the earlier Senior Design Project. The entire physical assembly, as tested, is described below.

1. Power System

There are 4S2P 18650 lithium-ion battery holders wired in parallel and mounted on the lower deck of the chassis, forming the main battery pack with a nominal voltage of 14.4 V and a fully charged voltage of about 16.8 V. The pack output is connected to an XL4015 step-down buck converter, which controls the voltage down to 12 V to drive the motor driver ICs, and to a 5 V regulator to power the sensors and the motors' encoders. To power the Raspberry Pi 5, a dedicated USB power bank is mounted separately on the chassis, keeping it completely isolated from the motor power domain and protecting it against voltage changes caused by inrush current during direction changes. Table 4-1 summarizes the estimated capacity, power consumption, and runtime of the main 4S2P battery pack used for the motors and Arduino-side electronics.

Table 4-1: Main 4S2P Battery Pack Capacity, Energy, and Estimated Runtime Calculation

Parameter	Value / Calculation
Single-cell capacity	2500 mAh
Total pack capacity	$2 \times 2500 \text{ mAh} = 5000 \text{ mAh} = 5 \text{ Ah}$
Voltage range	Nominal = $\sim 14.4 \text{ V}$ Fully charged = $\sim 16.8 \text{ V}$ Empty = $\sim 12.0 \text{ V}$
Total energy capacity	$14.4 \text{ V} \times 5 \text{ Ah} = 72 \text{ Wh}$
Motor current	$4 \times 102 \text{ mA} = 408 \text{ mA}$ (No load) $4 \times 304 \text{ mA} = 1.216 \text{ A}$ (Max. load)

Motor power	$12\text{ V} \times 0.408\text{ A} = 4.90\text{ W}$ (No load) $12\text{ V} \times 1.216\text{ A} = 14.59\text{ W}$ (Max. load)
Arduino Mega, 3 ultrasonic sensors, NeoPixel strip, and buzzer	$\approx 1.62\text{ W}$ (Avg.)
Total main battery power	$4.90\text{ W} + 1.62\text{ W} = 6.52\text{ W}$ (No load) $14.59\text{ W} + 1.62\text{ W} = 16.21\text{ W}$ (Max. load)
Theoretical runtime	$72\text{ Wh} \div 6.52\text{ W} = 11.0\text{ hours}$ (No load) $72\text{ Wh} \div 16.21\text{ W} = 4.4\text{ hours}$ (Max. load)
Practical runtime estimate	$4.4\text{ h} \times 0.85 \approx 3.8\text{ hours}$ (Avg.)

Table 4-2 summarizes the estimated capacity and runtime of the separate USB-C power bank used to power the Raspberry Pi 5 and USB camera.

Table 4-2: Raspberry Pi Power Bank Capacity, Energy, and Estimated Runtime Calculation

Parameter	Value / Calculation
Power source	10,000 mAh USB-C power bank
Maximum output power	30 W
Estimated stored energy	$10\text{ Ah} \times 3.7\text{ V} = 37\text{ Wh}$
Estimated usable energy	$37\text{ Wh} \times 0.85 \approx 31.45\text{ Wh}$
Worst-case Pi power estimate	$5.1\text{ V} \times 5\text{ A} = 25.5\text{ W}$
Worst-case runtime	$31.45\text{ Wh} \div 25.5\text{ W} \approx 1.2\text{ hours}$
Practical runtime estimate	$31.45\text{ Wh} \div 15\text{ W} \approx 2.1\text{ hours}$

The estimated operating time of the complete prototype is determined by the power source with the shortest operating time. The main 4S2P battery pack that powers the motors and Arduino-side electronics is estimated to provide about 3.8 hours of practical use, and the 10,000 mAh power bank for the Raspberry Pi 5 and USB camera is estimated to provide

about 2.1 hours during normal operation. Hence, the total prototype runtime is estimated at approximately 2.1 hours when both power sources are fully charged.

2. Motor System

Four DC motors (front-left, front-right, rear-left, and rear-right) are each connected to one output of two L298N dual H-bridge modules. The front pair of motors is driven by FRONT_DRV and the rear pair by BACK_DRV. The direction of each motor is controlled by a pair of digital output pins (IN1/IN2 pairs) and its speed by the ENA/ENB PWM enable pins, all connected to the Arduino Mega 2560. These are abstracted by the mecanum_driver library into clockwise and counter-clockwise movement commands per motor.

3. Arduino Mega 2560 Wiring

The main hardware controller is the Arduino. The connections of all the peripherals are as follows:

- Front ultrasonic sensor (HC-SR04): TRIG → pin 28, ECHO → pin 29
- Left ultrasonic sensor (HC-SR04): TRIG → pin 24, ECHO → pin 25
- Right ultrasonic sensor (HC-SR04): TRIG → pin 26, ECHO → pin 27
- NeoPixel LED strip (30 WS2812 LEDs): Data → pin 6, driven via the Adafruit NeoPixel library
- Piezo buzzer: Signal → pin 7
- Battery voltage sense: Analog pin A0, connected to the pack voltage range (12-16.8 V) via a 33 k Ω / 10 k Ω resistive voltage divider, which scales the pack voltage range (12-16.8 V) down to within the Arduino 5 V ADC range. The conversion is performed after averaging the six ADC samples to reduce noise
- Serial connection to Raspberry Pi: USB cable at 115,200 baud

The median of three successive pulses separated by 700 μ s is used to reject spurious echoes in each ultrasonic sensor reading. The sensor ranges are verified every 45 ms, and any forward motion command is overridden when the front range is found to be at or below 38 cm, a range determined to provide sufficient stopping margin at the rover's maximum

following speed. The Arduino implements an extra safety net that changes the command to stop if no new commands are received from the Raspberry Pi for more than 160 ms, as an additional safeguard against slow or missing serial messages.

4. Raspberry Pi 5 and Camera

Raspberry Pi 5 is attached to the upper deck of the chassis and connected to the Arduino via USB. The USB camera is placed at the front of the rover and set to capture at 480x270 pixels at 60 FPS using the V4L2 backend with MJPEG encoding and a buffer size of 1, minimizing capture latency. The Pi communicates with the Arduino over a USB serial connection at 115,200 baud, sending single-character motor-control messages and receiving battery telemetry in the format BAT, voltage, percentage every second.

Figure 4-1 shows the final prototype of the person-following rover.



Figure 4-1: Final Assembled Prototype of the Person-Following Rover

4.1.2 Software Setup

Raspberry Pi runs Raspberry Pi OS (64-bit) and Python 3.11. The major software requirements include Ultralytics YOLOv8 to detect persons and integrate with ByteTrack to track the detected object, MediaPipe to detect hand landmarks, OpenCV to capture and process images, and the requests library to communicate with the dashboard via HTTP.

The YOLOv8n model was exported to ONNX format and loaded on startup as `yolov8n_288.onnx`, with the input size of 288×288 to match the inference configuration and make it faster to run on the Raspberry Pi 5. Detection is computed at a 288×288 pixel input resolution, downsampled from the capture resolution of 480×270 , which speeds up system response time and reduces CPU usage. Detection is computed every frame in the LOCKED state and every 3 frames in the FREE state. MediaPipe hand processing runs on selected frames scaled to 35% of their original resolution. All these optimizations ensure that the main processing loop runs at around 15-20 FPS on the Pi 5, which is sufficient for reliable real-time following.

The Arduino has a 160 ms command timeout, which means that if it does not receive a new command from the Raspberry Pi within 160ms, it switches the command to stop. This will ensure that the rover will not start moving if serial communication is slow or interrupted. The sensor ranges are monitored on the Arduino at 45 ms, ensuring the obstacle detectors remain responsive regardless of the Pi's processing load.

The dashboard communication is executed on a background thread, which can be accessed through the domain <https://ff15.cam>. The Raspberry Pi polls pending commands at 250 ms, sends heartbeats at 10-second intervals, pushes telemetry, such as battery voltage, battery percentage, and CPU temperature, at 60-second intervals, and streams live video as a stream of compressed JPEG frames. Authentication is based on an API token stored in a local `.env` file, along with the rover ID and dashboard URL.

Figure 4-2 shows the dashboard interface containing telemetry information, live video, rover status, and manual control features.

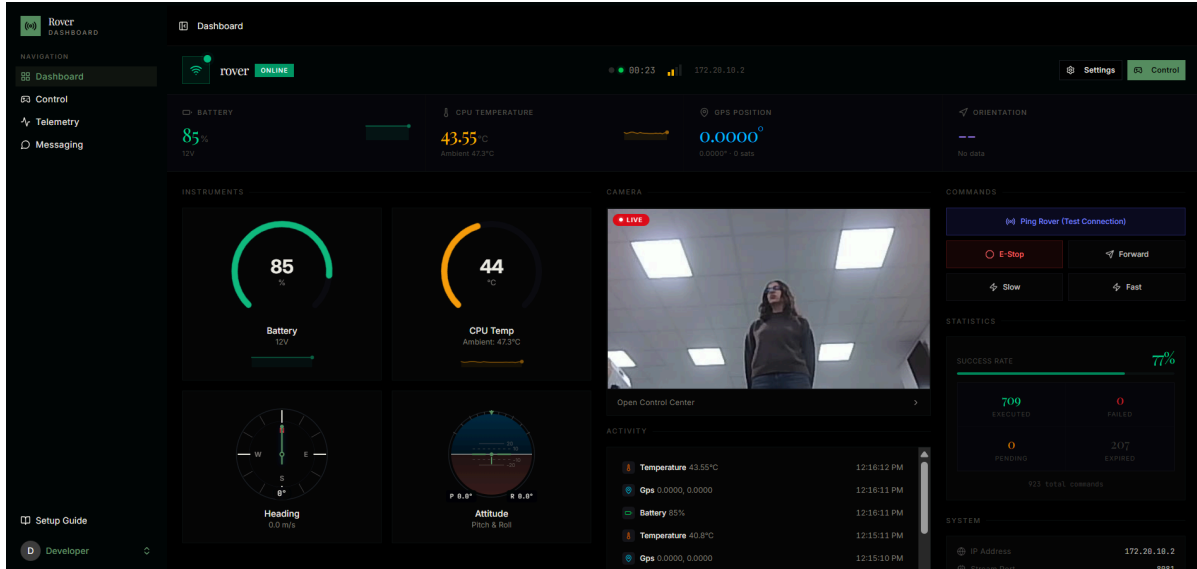


Figure 4-2: IoT Dashboard Interface containing Telemetry

The Arduino firmware is compiled and flashed on the Arduino IDE with the mecanum_driver and Adafruit_NeoPixel libraries installed. The serial interface is set up to 115,200 baud in setup(). The primary loop() executes sensor updates after every 45 ms, battery telemetry after every 1,000 ms, and implements the most recent Pi command on each iteration.

The complete control behavior, including the rotation pulse duration scaling between 15 ms to a far target and 250 ms to a close target, a 180 ms settle period between rotation pulses, a 190-pixel bounding box height stop threshold, and a 18% horizontal deadzone, was detailed in Section 3.5 and applies directly to the prototype as tested.

4.2 Testing

The system was tested using a set of structured tests designed to assess a particular design requirement or behavioral aspect of the rover. All tests were performed on a flat, indoor floor under normal indoor lighting. It is a complete software stack running on the rover, with no parts disabled during testing.

Test 1: Person Detection and Lock Acquisition

Table 4-3: Person Detection and Lock Acquisition Test

Item	Description
Purpose	Check that the rover is properly configured to detect a person and switch from FREE to LOCKED state when a valid peace-sign gesture is performed.
Setup	The rover was set up in a single stationary position, with a person standing at different distances from the camera. At each distance, the individual raised a peace sign and held it inside of their bounding box for the required gesture-detection time.
Expected results	The rover enters the LOCKED state, the NeoPixel strip turns solid green, and the bounding box of the correct person is highlighted in the live feed within a reasonable time after the gesture is held.

Test 2: Person Following at Various Speeds

Table 4-4: Person Following at Various Speeds Test

Item	Description
Purpose	Check that the rover continuously follows a locked target at a minimum speed of 0.5 m/s.
Setup	Once the target was locked on, the individual would walk away in a straight line in front of the rover. The target walks at a walking speed. Another series of runs involved a mid-path lateral change in direction to test how the rover would respond to a target that suddenly changed direction.
Expected results	The rover follows without losing lock, maintains a safe following distance, and stops automatically when the person's height becomes 190-pixels on the camera.

Test 3: Rotation and Lateral Tracking

Table 4-5: Rotation and Lateral Tracking Test

Item	Description
Purpose	Check that the rotation command logic is correct, that it centers the target when it moves laterally across the frame, and that unnecessary micro-rotations are suppressed by the deadzone.
Setup	The target moved laterally across the rover's field of view without walking directly away. The individual stood on the left side of the frame, remained still, then moved to the right side at different distances to test the pulse duration/speed and ability to center the target.
Expected results	The rover picks the right rotation direction and speed level (Q/W/E or I/O/P), the target returns to the deadzone, and no unnecessary micro-rotations are experienced when the target is roughly centered.

Test 4: Obstacle Avoidance

Table 4-6: Obstacle Avoidance Test

Item	Description
Purpose	Check that the Arduino-layer obstacle override stops the rover when the front sensor reads 38 cm or less, regardless of the active command of the Pi.
Setup	An obstacle was placed between the rover and a locked target, about 30 cm ahead of the rover's front, with the Pi actively transmitting forward commands. In another series of experiments, the obstacle was removed in the middle of the test to ensure the rover could resume forward movement properly.
Expected results	The rover halts before contact in all trials. When the rover stops, the front sensor readings give around 38 cm or slightly less, and the rover begins moving again when the obstacle is cleared.

Test 5: Target Loss and Recovery

Table 4-7: Target Loss and Recovery Test

Item	Description
Purpose	Test the entire LOST state behavior, including timeout transitions, LED and buzzer feedback, strafe behavior after 5s, and return to FREE after 15s.
Setup	Once the target is locked, it leaves the field of view of the camera and remains out of view throughout the entire duration of the test. The state transitions to the LOST state, the flashing LED and buzzer pattern, the strafe direction decision based on the sensor readings on the left and right sides, and the transition back to the FREE state were all observed and timed.
Expected results	The system switches to LOST when the target leaves the camera view, the visual and audio feedback activate properly, the rover strafes to the side with the smaller distance reading after 5 seconds, and returns to FREE after 15 seconds without reacquisition.

Test 6: Manual Override through Dashboard

Table 4-8: Manual Override Through Dashboard Test

Item	Description
Purpose	Check that the dashboard manual override is functional and that the rover can switch between autonomous following and manual control.
Setup	When the rover was actively following a locked target, the user switched to manual control from the dashboard. Directional commands were given one at a time: forward, left, right, and stop. Then the user switches from manual control to autonomous mode.
Expected results	The rover instantly suspends autonomous following on the receipt of the override, executes each manual directional command correctly, and returns cleanly to FREE state on the issuance of the auto-follow.

Test 7: Battery Life

Table 4-9: Battery Life Test

Item	Description
Purpose	Confirm that the rover can run at least one hour continuously with both power sources fully charged.
Test Setup	The rover batteries were fully charged before the test, then ran continuously in person-following mode with the dashboard connected and video streaming. The time was recorded as the time taken between the startup and the point where the rover exceeded the required 1-hour operation time, without fully discharging the batteries.
Expected results	Total run time is equal to or greater than 60 minutes.

Test 8: IoT Telemetry and Dashboard Communication

Table 4-10: IoT Telemetry and Dashboard Communication Test

Item	Description
Purpose	Check that heartbeats, telemetry, and video reach to the dashboard at the desired rates, and that manual commands are delivered within an acceptable latency range.
Test Setup	The dashboard was observed for 10 minutes of active rover operation. Timestamps for heartbeat reception, telemetry updates, video stream refreshes, and the time interval between dashboard command issuance and the rover's physical response were recorded.
Expected results	Heartbeats are received every 10 seconds, telemetry updates are received every 60 seconds, the video stream is updated continuously, and dashboard commands are executed with an observable rover response within one polling cycle (250 ms).

4.3 Prototype Testing Results Discussion

Experiment 1: Person Detection and Lock Acquisition

The camera feed was captured successfully, and YOLOv8n was able to produce person bounding boxes consistently under typical indoor lighting, as shown in Figure 4-3. The peace-sign gesture was consistently detected at distances of 0.4-1.0 m when the hand was visible, while the open-palm gesture was used to release the target, as shown in Figure 4-4. The success rate decreased when the target was far away because the hand landmarks were less visible to detect and resolve at the 35% processing scale applied in MediaPipe. Within the reliable range, the rover successfully switched to the LOCKED state, and the NeoPixel strip remained solid green throughout all successful locking operations. Figure 4-5 shows how the target's position inside the camera frame was translated into movement commands such as rotate left, rotate right, stop, and forward.

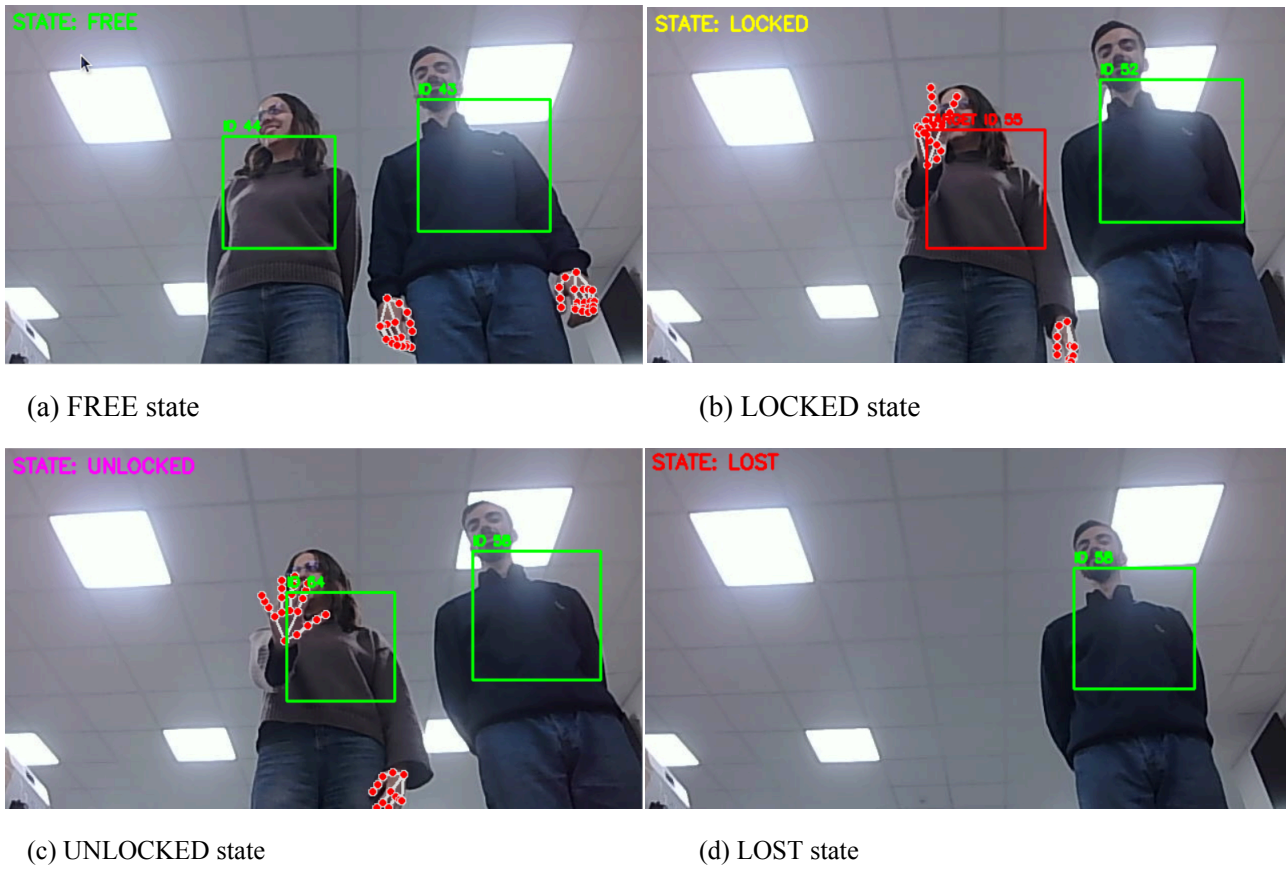
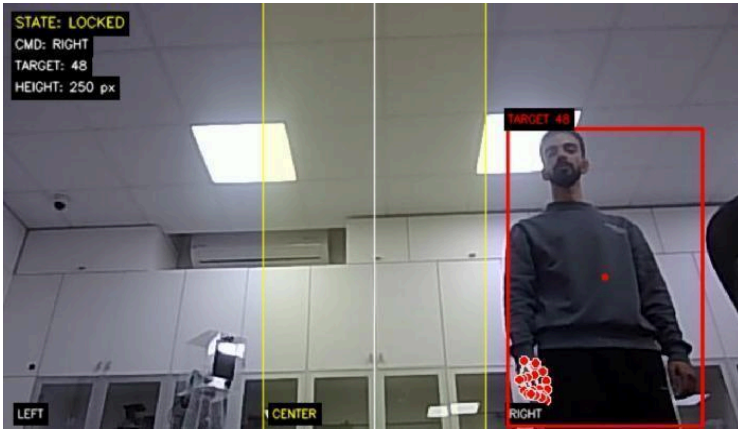


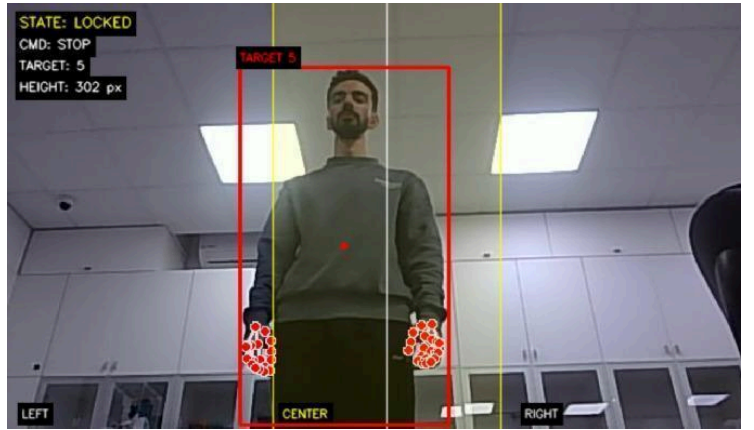
Figure 4-3: Camera Detection and Tracking Output During Target Locking



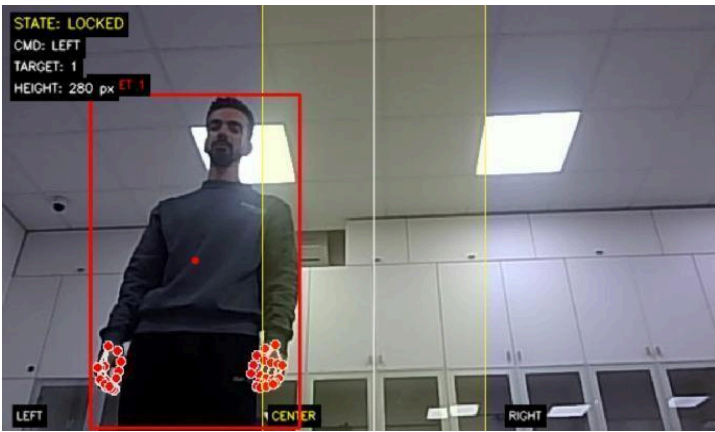
Figure 4-4: MediaPipe Hand Landmark Detection



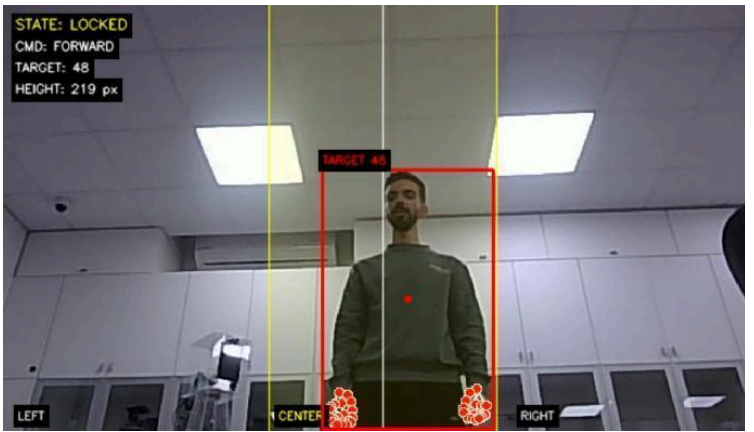
(a) Target on the left side / rotate left command



(b) Target too close / stop condition



(c) Target on the right side / rotate right command



(d) Target centered and far / forward command

Figure 4-5: Target Visibility and Frame-Margin Conditions with Corresponding Rover Commands

Experiment 2: Person Following at Different Speeds

The rover was able to walk through all straight-path runs without losing lock. The loss of lock was occasionally observed when the target was partially visible, caused by objects or people walking around, or when the target performed a sharp turn, briefly pushing the target out of frame before the rover could rotate to compensate. The three-tier rotation speed system, LOW, MED, and HIGH, generated smooth recentering behavior during the following. The settling period (180 ms) between rotation pulses did not allow oscillation without creating a noticeable delay that affects the following procedure. The 190 pixel

height threshold correctly activated a stop command when the target was too close, and forward motion was correctly restarted when the target height was lower than 190 pixels.

Experiment 3: Rotation Tracking

The rotation command logic worked well in all the subsequent left-right tracking trials. As the target moved to one side or the other of the frame, the rover chose the appropriate rotation direction and speed based on the size of the horizontal error. The 18% dead zone on either side of the frame center successfully prevented unnecessary micro-rotations when the target was roughly centered. The scaling of the rotation pulse duration with target height was also confirmed; the rover rotated with shorter pulses on distant targets and longer pulses on closer targets, producing significantly smoother correction at varying ranges. However, in some cases where the target was very close to the rover and moved rapidly around it, caused the system to lose the target as the rover could not rotate and recenter the target in time.

Experiment 4: Obstacle Avoidance

The 38 cm forward stopping threshold worked well in all the trials. The Arduino overrode the active forward command and stopped the motors before any contact with the obstacle was made. The front sensor reading also confirmed that at the point of stopping in each trial, the measured distance from the rover to the obstacle was 38 cm or slightly below that. The three-sample median filter was able to reject spurious echo readings - no false triggers were seen in any trial. When the obstacle was moved during the test, forward motion was once again correct in all cases. But a limitation was noted when an object was brought very near to the ultrasonic sensor. The sensor returned a false value of 999 cm, a non-representative or out-of-range value rather than the actual distance. This implies that the ultrasonic sensor may become unreliable when the obstacle is too close to the sensor.

Experiment 5: Target Loss and Recovery

When the locked target left the camera frame, the rover correctly stopped and entered the LOST state. The pattern of flashing green LED and double-beep buzzer were activated as expected in all trials. The rover read the ultrasonic sensor on the left and right sides and strafed towards the nearest object/wall. In all trials, the system was left in the FREE state after the full 15-second LOST timeout if the target was not reacquired. Reacquisition, however, only worked when the target returned to the camera frame within a range of about one second after the target was lost. The system was typically unable to automatically reacquire the original target after this short period, which means a new lock gesture would be needed to reacquire the original target.

Experiment 6: Manual Override by Dashboard

Dashboard manual override became effective in all experiments. Autonomous following was suspended in the rover when the override command was sent and responded appropriately to each directional input, forward, left, right, and stop, issued by the dashboard. However, a response delay was observed since the dashboard is hosted on a cloud service, which means that the commands must travel the network before reaching the rover. The switch back to autonomous mode via the auto-follow command returned the system to the autonomous state in all trials, and the gesture lock mechanism was immediately ready to accept a new target.

Experiment 7: Battery Life

The rover operated successfully for over 1 hour while running the following system and dashboard functionality. The battery voltage and calculated battery percentage were shown on the dashboard during the test as it steadily declined over the course of the test. As the rover was still running by the time it had passed the 1-hour mark, the test confirmed that the system meets the minimum battery-life requirement. Nevertheless, the maximum operating time was not measured as the battery was not completely discharged during the experiment.

Experiment 8: IoT Telemetry and Dashboard Communication

The dashboard received heartbeats at 10-second intervals during the monitoring session, with no missed beats. Telemetry battery voltage, percentage, and CPU temperature are updated every 60 seconds, as configured. The video stream showed frames at a steady rate of about 8-12 FPS, depending on network conditions, which is adequate for remote monitoring applications. However, about 5 seconds of delay were experienced in the video stream because of cloud service latency and the high bandwidth needed to transmit live video. The dashboard commands generated visible rover reactions in a single polling cycle. Figure 4-6 shows the dashboard interface during rover operation, including the live camera view, command controls, telemetry information, and system status.

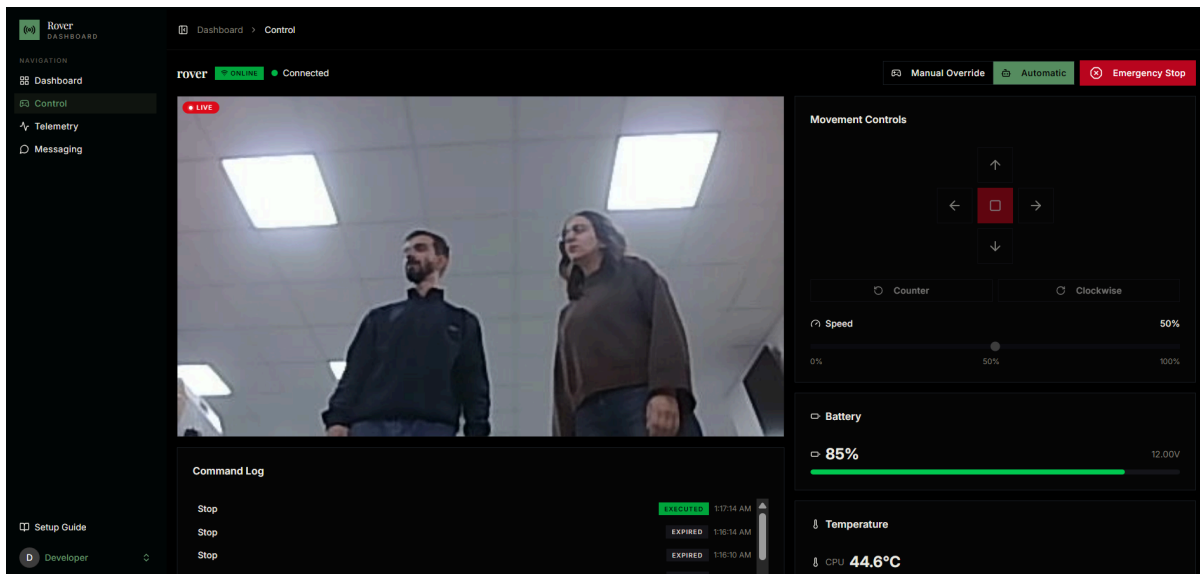


Figure 4-6: IoT Dashboard Interface During Rover Operation

4.4 Validation of Design Requirement within the Realistic Constraints

Table 4-11 below summarizes the validation of each design requirement and realistic constraint against the test results.

Table 4-11: Validation of Requirements and Design Constraints

	Parameter	Required	Achieved	Explanation
Requirement 1	Person tracking speed	≥ 0.5 m/s	✓	Constant following at walking speed with reliable continuity in all the runs. There were occasional losses of lock on sharp lateral turns at higher speeds.
Requirement 1	Real-time detection	OpenCV /YOLOv8	✓	YOLOv8n with ByteTrack on the Raspberry Pi 5 using reduced resolution and frame skipping to increase responsiveness.
Requirement 2	Cloud data transmission	Telemetry to the server	✓	Battery voltage, percentage, and CPU temperature are sent to the dashboard every 1 minute.
Requirement 2	GPS data transmission	Location data to the server	✗	GPS was not implemented because the rover is mainly intended for indoor use, where GPS signals are weak and unreliable.
Requirement 2	Remote control and monitoring	Web interface	✓	The dashboard manual override, live video stream, and rover state display are all functional, but with a delay.
Requirement 3	Obstacle detection	Avoid obstacle collision	✓	Arduino override stopped the rover when close obstacles were detected, with minimal false triggers.
Requirement 3	Safety independence	Local safety regardless of the network	✓	Obstacle avoidance and target loss caused the rover to stop without a network connection.
Requirement 4	Battery life	≥ 1 hour	✓	Rover successfully operated for over 1 hour.
Economic constraint	Total cost	< 500 JD	✓	All components were obtained within the project's budget.
Environmental constraint	Battery safety	IEC 62133 considered	✓	Voltage monitoring is active through the dashboard; every 4 cells are installed separately.
Manufacturability constraint	Component availability	Off-the-shelf only	✓	All components are available on the market.

5 Conclusion and Future Work

5.1 Conclusion

The primary objective of this project consisted of designing and implementing an IoT-enabled rover that has the capability to detect, lock onto, and follow a chosen person and provide remote monitoring and control via a dashboard. The system's prototype consists of computer vision, motion control, obstacle detection, battery monitoring, and IoT communication.

The final prototype demonstrated the key functionality. The rover was able to detect a person, lock onto the selected target using a specific hand gesture, follow the target during straight-path and left-right tracking tests, stop when the target became too close, enter a LOST state when the target left the camera frame, and unlock the target when an open-palm gesture is detected. The dashboard also offered some useful monitoring options, such as battery percentage, CPU temperature, video streaming, heartbeat messages, switching to manual mode to send manual control commands, and switching back to autonomous mode when needed.

The project also demonstrates that a low-cost embedded platform can be utilized to create a person-following rover with IoT monitoring capabilities. The use of a Raspberry Pi, Arduino Mega, camera, ultrasonic sensors, and a cloud-hosted dashboard provided a practical implementation of a person-following robotic system. The system also demonstrated that vision-based tracking is important when combined with safety features such as obstacle detection, stopping thresholds, and manual override.

5.2 Future work

In future work, the system can be enhanced in several ways. To minimize video streaming delay on the dashboard, a better cloud service or a local edge-streaming solution should be implemented. Indoor localization can be implemented using other localization methods rather than GPS, which are more appropriate for indoor environments, such as ultra-wideband (UWB) anchors, Bluetooth beacons, Wi-Fi positioning, visual SLAM, or fiducial markers like AprilTags. The obstacle detection system can be enhanced so the rover does not collide with an obstacle on its edges by increasing the number of sensors or using different sensors, like LiDAR, depth cameras, or time-of-flight, to provide a better coverage around the rover's body. A stronger target reacquisition technique can be implemented to re-identify the locked target when re-entering the frame after longer periods of loss. Moreover, the processing speed for detection, tracking, and motion control could be improved by using a more powerful processor or a dedicated AI accelerator, rather than relying on the Raspberry Pi 5. It might also be studied in future research how the concept of multi-person tracking, enhanced gesture recognition, sensor fusion between vision and distance sensors, and more efficient edge-AI models could be used to achieve real-time autonomous navigation.

References

- [1] International Federation of Robotics, “Service Robots See Global Growth Boom,” IFR, Oct. 2025. [Online]. Available: International Federation of Robotics website. [Accessed: May 2026].
- [2] A. Eirale, M. Martini, and M. Chiaberge, “Human Following and Guidance by Autonomous Mobile Robots: A Comprehensive Review,” *IEEE Access*, vol. 13, pp. 42214–42253, 2025, doi: 10.1109/ACCESS.2025.3548134.
- [3] I. Condés, J. Fernández-Conde, E. Perdices, and J. M. Cañas, “Robust Person Identification and Following in a Mobile Robot Based on Deep Learning and Optical Tracking,” *Electronics*, vol. 12, no. 21, Art. no. 4424, 2023, doi: 10.3390/electronics12214424.
- [4] Z. Huang, C. Ou, Z. Guo, L. Ye, and J. Li, “Human-Following Strategy for Orchard Mobile Robot Based on the KCF-YOLO Algorithm,” *Horticulturae*, vol. 10, no. 4, Art. no. 348, 2024, doi: 10.3390/horticulturae10040348.
- [5] P. Rakprayoon, A. Sudsang, and T. Maneewarn, “A Cognitive Robotic System for a Human-Following Robot,” in *Fuzzy Systems and Data Mining VI*, 2020.
- [6] A. Boschi, M. Garabini, A. Catalano, G. Grioli, and A. Bicchi, “A Cost-Effective Person-Following System for Assistive Unmanned Vehicles with Deep Learning at the Edge,” *Machines*, vol. 8, no. 3, 2020.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proc. European Conference on Computer Vision (ECCV)*, 2022, pp. 1–21, doi: 10.1007/978-3-031-20047-2_1.
- [9] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device Real-time Hand Tracking,” *arXiv:2006.10214*, 2020.
- [10] V. Kamath and A. Renuka, “Investigation of MobileNet-SSD on Human Follower Robot for Stand-Alone Object Detection and Tracking Using Raspberry Pi,” *Cogent Engineering*, vol. 11, no. 1, 2024, doi: 10.1080/23311916.2024.2333208.
- [11] SparkFun , “Voltage Dividers,” SparkFun Learn. [Online]. Available: <https://learn.sparkfun.com/tutorials/voltage-dividers/all>. [Accessed: May 2026].

Appendices

Appendix A: Offering Template

Note: The original project offering included object/person following. In this implementation phase, the project scope was focused on person following, since the developed prototype uses camera-based person detection, gesture-based target locking, and autonomous following of a selected person.



King Abdullah II
School of Engineering
كلية الملك عبد الله الثاني
للهندسة

Princess Sumaya University for Technology King Abdullah II School of Engineering Senior Design Projects

Design and Implementation of an IoT-Enabled Rover for Person Following

1	Advisor Name	Prof. Esam Qaralleh
	Co-Advisor Name	
2	Year / Semester	Fall 2025/2026
3	Title of Senior Design Project	Design and Implementation of an IoT-Enabled Rover for Person Following
4	Design to be Achieved	This project focuses on designing and building a rover robot that can autonomously follow an object or person using IoT-enabled remote control and monitoring. The rover will use cameras and sensors to detect and track a target in real-time, sending relevant data such as location and video stream to a cloud server for remote monitoring and control. Hint: The rover used in this project is the design outcome of a previous SDP by the students Ahmad Mwalla, Rama Abdelhadi, and Mohammad Fayoumi. Project Components: • Rover Robot with Object/Person Tracking: • A four-wheel rover robot capable of following a moving object or person based on visual and sensor data. • Integrated cameras and sensors for object/person detection. • Autonomous navigation system using obstacle avoidance technology. • IoT Integration for Remote Control and Monitoring: • Real-time communication with a cloud platform for monitoring rover status, tracking data, and remote control capabilities. • Web or mobile interface to control the rover, view video feed, and adjust tracking parameters. • Autonomous Features: • The rover should be capable of identifying and following a specific target without manual intervention. • Ability to navigate safely around obstacles while tracking the target.

5	Engineering Standard	IEEE 802.11 – Wi-Fi Standards IEC 62133 – Safety Requirements for Portable Battery Systems	
6	Design Requirements	• Real-time Object/Person Tracking: • The rover shall process video feed from an onboard camera to detect and track a designated object or person. • The system shall be able to follow the target at a speed of at least 0.5 m/s with minimal delay. • The design shall utilize image processing techniques (e.g., OpenCV) to identify and track objects in real time. • IoT Communication: • Data such as the rover's current location, target position, and sensor readings shall be sent to a cloud server. • The system shall support remote control and monitoring through a web or mobile interface. • The IoT platform shall allow users to receive alerts or control the rover manually if needed (e.g., via MQTT or HTTP protocols). • Obstacle Detection and Avoidance: • The rover shall detect and avoid obstacles using sensors such as ultrasonic or LiDAR. • The system shall prioritize safety by stopping or navigating around obstacles while tracking the target. • Energy Efficiency and Battery Life: - The rover shall operate for a minimum of 1 hour on battery power. - The design shall optimize power consumption by using low-power sensors and efficient processing algorithms.	
7	Realistic Constrains	Economic	- The cost of the design shall be less than 500 JD.
		Environmental	Battery Type: The choice of batteries (e.g., lithium-ion, nickel metal hydride) should consider environmental impact, including toxicity and recycling options. Rechargeable and long-life batteries reduce waste and the need for frequent replacements.
		Manufacturability and Sustainability	
		Other	
8	Deliverables	Documentation	Yes
		Portfolio	Yes
		Video	Yes
		Other	• Code • Schematics • Mechanical Design files • Running prototype • Operation manual • GitHub link with all required deliverables
9	Background of Students		
10	Number of Students	3	

Advisor Signature

Co-Advisor Signature

Department Head Signature

Appendix B: Code

The GitHub link below includes the following: Arduino code, Raspberry Pi Code (Camera and dashboard), rover schematic, system flowchart with its PlantUML code, and the project's documentation.

Repository link: <https://github.com/hamztion/Person-following-Rover-Project.git>

Rover's Dashboard link: <https://ff15.cam>